INDEX

8.1

NAME: Bhuvanesh M STD.: _____ SEC.: _____ ROLL NO.: 240701081 SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	20/7/26	Basic Networking Commund	8	
2.	28/7/26	Cisco packet Tracer	8	
3.	27/7/26	Study of Network cable	8	
4.	3/8/26	Setting up a Local area Network	9	
5.	10/8/26	Packet captue Tool: Wireshark	9	
6.	27/8/26	Error detection & Error Correction	9	
7.	30/8/26	Sliding window protocol	9	
8.	2/9/26	NMAP to discover live host using Nmap scans	8	
9.	7/9/26	Implementation of subnetting in cisco packet Tracker Simulator	8	
10.	10/9/26	Internetworking with router in CISCO packet tracker	9	
11.	15/9/26	Routing at network layer	10	
12.	23/9/26	End to end communication at transport layer	10	
13.	29/9/26	Implement your own ping program.	10	
14.	2/10/26	Raw sockets. Packet Sniffing	10	
15.	5/10/26	Analysis of web logs using verbalizer tool	10	

Experiment - 1

Basic Networking Commands

AIM:

Study of various network commands used in Linux and windows.

PROCEDURE:

1. Start the computer login to the windows / Linux operating system.
2. Open the PUTTY application on the windows system
3. Enter the IP address on the Linux server and establish an SSH connection
4. Login in using the valid username and password.
5. Execute the required N/W commands.
6. Observe the O/P displayed for each command

DESCRIPTION

Commands I/P & O/P

① arp -a

O/P: Displays IP & MAC address table

Observed: ARP cache entries were displayed successfully

② hostname

O/P: DESKTOP-ABC123

Observed: The system hostname was displayed.

③ ipconfig

O/p : IP address, subnet Mask, Gateways, DNS

Observed : Network configuration details were displayed.

④ nbtstat -a <hostname>

O/p : Net BIOS nametable

Observed : Active network connections and routing information were displayed.

⑤ netstat -r

O/p : Routing Table

Observed : routing & Connection information was displayed.

⑥ nslookup www.google.com

O/p : DNS server and IP address

Observed : DNS name resolution was verified successfully.

⑦ Pathping www.google.com

O/p : HOP details and packet loss

Observed : The network path and packet loss statistics were analyzed.

⑧ ping www.google.com

O/P: Reply from I/P with time

Observed: Network connectivity was verified successfully.

⑨ route print

O/P: Routing table entries.

Observed: The system routing table was displayed

⑩ ip address show

O/P: Interface details with I/P address

Observed: I/P assigned to the network interface was displayed.

RESULT: 8/10

The Basic networking commands were executed successfully.

EXPERIMENT - 2

CISCO Packet Tracer

AIM:

To study the packet tracer tool installation and user interface

DESCRIPTION:

Thus experiment introduced cisco packet tracer simulation tool and its user interface for designing and simulation computer network. It also covers basics networking device such as hubs and switches A hub broadcasts data to all connected device, whereas a switch forwards data only to the intended destination using mac address making communication faster and more efficient.

PROCEDURE:

① Install & launch cisco packet Tracer

② Observe the user interfaces including the device toolbar and workspace.

③ Drag and place networking device such as PCs, hubs and switches onto the workspace.

④ Connect the device using appropriate tables.

⑤ Assign IP address to the PCs.

⑥ Test connectivity between the device using ping command.

⑦ Observe the data transmission through the hub and the switch

⑧ Save the network topology and record observation.

OBSERVATION:

(a) Behaviour of switch and hub in forwarding packet

Hub: A hub forwards the received data to all connected device without checking destination address.

Switch: A switch forwards the received data only to the destination device by using MAC address, making communication faster and secure

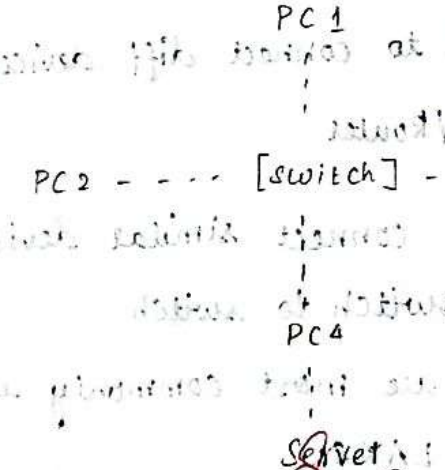
(b) Network topology implemented in the clg

Most colleges use star topology where all computer are connected to central switch.

The hub broadcast packet to all connected device.

The switch forwards packet only to the intended destination using mac address

DIAGRAM:



RESULT:

The cisco packet Tracker environment was successfully explored. The working of hubs and switch was studied and verified.

EXPERIMENT - 3

Study of Network Cables

AIM:

Understand & study of different types of network cables.

DESCRIPTION:

This experiment was conducted to study the construction and use of straight through and crossover twisted pair cables. The color coding standards were followed to prepare the cables using RJ45 connectors and a crimping tool. The prepared cables were tested to verify proper network connectivity b/w devices.

1) Difference b/w cross cable & straight cable

Straight cable: used to connect diff devices

ex: PC to switch / Router

Cross cable: used to connect similar device

ex: PC to PC or switch to switch.

The straight cables are most commonly used ethernet cable in LANS.

2) The type of cable is used to connect two PCs are cross cable. The cross cable is commonly used to connect PC to PC, switch to switch or Router to Router.

3) Type of cable used to connect a router / switch to your PC.

4) The category of twisted pair cable in the lab to connect the PC to the CAT 6 UTP- category 6 unshielded twisted pair cables used lab.

5) Learned to arrange wires in the correct T568A / T568B color sequence and crime the RJ45 connector.

6) Challenges faced is where arranging the wires correctly and crimping the connectivity properly.

7) The cable was successfully proved and network connectivity was verified using cable tester.

RESULT:

The straight & crossover cables were successfully prepared and tested.

EXPERIMENT-1

Setting up a local area Network

AIM:

To design, set up and configure a local area network connecting 4 computers through a network switch, assign static IP address to all devices verify connectivity using ping command and demonstrate data sharing across the network.

IP configuration table:

Device	IP address	Subnet mask	Connected to
PC0	125.1.1.0	255.0.0.0	Switch Fa0/1
PC1	125.1.1.1	255.0.0.0	Switch Fa0/2
PC2	125.1.1.2	255.0.0.0	Switch Fa0/3
PC3	125.1.1.3	255.0.0.0	Switch Fa0/4
Switch	125.1.1.4	255.0.0.0	VLAN1 Interface

Implementation:

Step 1: Open cisco packet trace and place switch, 4 PCs and one server in the workspace.

Step 2: Connect all PC's and server to the switch using copper straight through cables.

Step 3: Configure all IP address of all PCs and the server so that they belong to same network.

Step 4: Verify all cable connection are active

Step 5: Test network connectivity, by opening the command prompt on a pc using ping command to communicate with another PC or server.

Step 6 : Enable the HTTP services on the server by selecting server → services → HTTP ON

Step 7 : Open web browser on PC and enters the server IP address to access the servers default web page.

Step 8 : Switch from realtime mode to simulation mode in packet tracer.

Step 9 : Send a ping or web request from a PC to the server and use the capture / forward or auto capture / play option to observe the packet travelling through the switch.

Step 10 : verify that successful communication taken place b/w the PC's and the server, demonstrating proper LAN operation and data sharing

RESULT:

A LAN was successfully established using switch and a server, communication b/w devices as verified through ping and web access and packet flow was observed successfully in simulation mode.

EXPERIMENT-5

Packet Capture Tool: Wireshark

AIM:

To capture and analyse network packet using wireshark packet capture tool and apply filters for TCP, UDP, DNS, HTTP etc.

Wireshark:

Wireshark a network analysis tool formerly known as ethereal, captures packets in real time and display them in human readable format, wireshark includes filters, color coding and other features that let you dig deep into network traffic and inspect a suspicious programs network traffic, analyze the traffic flow on network

Capture the packets:

Step1: Open the wireshark application

Step2: Select active network interface such as wifi or ethernet.

Step3: Double click the selected interface to start packet capturing.

Step4: Observe packet appearing in the packet list

Step5: Allow wireshark to capture packet for few sec.

Step6: Click the stop button to stop packet capturing

Step7: Observe the captured packet and their details

Step 8: Save the captured packet using file → Save as

Step 9: Take screenshot of captured packet

Filtering the packet

Step 1: Click the display filter box at the top of the wireshark window.

Step 2: Enter the required protocol filter

Step 3: Press enter to apply filter

Step 4: Wireshark displays only the packet belonging to selected protocol.

Step 5: Inspect the filtering packet in packet list and packet detail panes

Step 6: Repeat the process for different protocol.

RESULT:

The network packet were successfully captured and saved using wireshark the captured packet were successfully filtered based on diff: protocols.

EXPERIMENT - 6

Error detection & Error correction

AIM:

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

Sender program:

* Input to sender file should be a text of any length program should convert the text to binary.
* Apply hamming code concept on the binary data and add redundant bits to it.
* Save in a file called channel.

code (sender.py)

```
data = list (map(int, input ("Enter data bits : "). ship()))
m = len (data)
r = 0
while (2 ** r) < (m + r + 1):
        r + = 1
n = m + r
hamming = [0] * (n+1)
j = 0
for (i in range (1, n+1) :
    if (i & (i-1)) ! = 0;
        hamming [i] = data [j]
        j + = 1
for p in range (r):
        pos = 2 ** p
```

```
        parity = 0
for i in range (1, n+1):
    if i & pos:
        parity = hamming[i]
hamming[pos] = parity
code = hamming[1]
print("Data bits: data)
print("Hamming code: code)
```

Output:

```
Enter data bits: 1011
Data bits: [1, 0, 1, 1]
Hamming code: [0, 1, 1, 0, 0, 1, 1]
```

Reciever code (reciever.py)

```
recieved = list(map(int, input("Enter recieved
                                hamming code: ").strip()))
n = len(recieved)
r = 0
while (2** r) <= n:
    r+ = 1
error pos = 0
for p in range(r):
    pos = 2** p
    parity = 0
    for i in rang (1, n+1):
        if i & pos:
            parity ^= recieved[i-1]
        if parity != 0:
            error post += pos
```

```
    if error pos = 0;
        print ("No error detected")

    else:
        print ("Error detected at position; " error-pos)
        reieved [error.pos = 1] ^ = 1
        print ("Corrected hamming code:", received)
data []
  for i in range (1, n+1):
      if (i & (i-1)) != 0;
      data. append (recieved (i-1)]
       print ('Original data list : "data)
```

Output:

Suppose hL : 0110011 change to 0111011

Error detected at position : 4

Corrected hamming code : [0, 1, 1, 0, 0, 1, 1]

Original data bits : [1, 0, 1, 1]

RESULT:

Thus, the hamming code algorithm was successfully implemented in python, the sender generates the hamming code from the given data. Bits, the receiver successfully detected and corrected a single bit error.

EXPERIMENT - 7

Sliding Window Protocol

AIM:

To write a program to implement flow control at data link layer using sliding window protocol stimulate flow of frames from one node to another.

Sender code:

```
import time
import os
print ("=" * 55)
print ("Sliding window protocol - sender")
window_size = int (input ("Enter window size"))
if [window_size <= 0;
    print ("Invalid window size !")
    exit ()
message = input ("Enter text message !")
if not message
    print ("Message cannot be empty!")
    exit ()
frames = [ ]
for i, character in enumerate (message);
    frames, append ( { "frames - no"; i,
                       "data": character
        })
    print ("Frames created:")
    for frame in frames;
    print ({ frame no : { frame ["frame_no"]}
        DATA : { frame ['data']} ")
```

```
total_frames = len(frames)
base = 0
while base < label_frames
    end = min(base + window_size, label_frames)
    print("sending frames {base} to find {")
    with open("sender_buffer", w) as file;
        file i in range | base end()

try:
    with open("Receiver-Buffer", "r") as file;
        response = file.readline().strip()
    except FileNotFoundError;
        print("Receiver_buffer not found!")
    parts = response.split()
    if len(parts) != 2;
        print("Invalid Receiver-Buffer found!")
        continue;
    else:
        print("Invalid response received")
    time.sleep(1)
    print("All frames transmitted successfully")
print("original message {message}")
```

Output:

```
SLIDING WINDOW PROTOCOL SENDER
Enter window size: 3
Enter test message: HELLO
```

Sender Buffer

Frame No : 0 DATA : H

Frame No : 1 DATA : E

Frame No : 2 DATA : L

Receiver response : ACK 3

all frames transmitted successfully

original message : HELLO

Receiver code :

```
import OS
SENDER_FILE = "sender_buffer.txt"
RECEIVER_FILE = "receiver_buffer.txt".
expected frame = 0
while True:
    if NOT OS.path.exist (sender_file);
    print ("In frames received")
    error = false
    error_frame = None
else:
    with open (RECEIVER_file; "w") as file;
        file.write (f "ACK : {expected file}")
    print (f" {n ALL frames received correctly")
    print (f "ACK: {expected_frame}")
```

Output :

Frames Received

frame No: 0 Data: H

frame No: 1 Data: E

frame No: 2 Data: L

All frames received correctly

ACK : 3

Result!

Thus, the sliding window protocol was implement successfully using python and flow of frames between sender and receiver was simulated using ACK / NACK

EXPERIMENT - 8

NMAP TO DISCOVER LIVE HOSTS USING NMAP SCANS

AIM:

To discover live hosts on a network using Nmap host-discovery scans - ARP scan, ICMP scan and TCP/UDP ping scan - and to understand which protocol operates at which layer of TCP/IP model, when each one applies.

Nmap:

Nmap is a free open-source network scanning tool created by Gorden Lyon and first released in 1997.

Procedure:

1) Log into TryHackme and open the Nmap live host discovery room
2) Work through tasks 1-3 and answer the simulator based questions
3) Deploy the lab network in Task 4 and start the attackbox note target subnet.
4) Run in ARP scan. PR <subnet>/24 and record how many host respond.
5) Run an ICMP echo scan, sudo nmap -PE, result with ARP scan
6) Run ICMP timestamp, and ICMP address.
7) Run TCN SYN (PS) TCP ACK, UDP scan
8) Optionally, capture the scan traffic using topdump on wireshark on tuno interface.
9) Complete the knowledge check queue and proceed to Nmap Base port scan room

Result:

Thus Nmap host discovery scan were performed successfully using TCP, SYN, FCP, ACK UDP Ping technique, the scan result network traffic were observed.

EXPERIMENT - 9

Implementation of Subnetting in CISCO PACKET TRACKER Simulator

AIM:

To implement subnetting in cisco packet Tracker by creating multiple subnets, assigning static IP addresses connecting the subnets using router, testing packet transmission.

THEORY:

Subnetting is the process of dividing one network into smaller network called subnets. It helps in efficient use of IP addresses and allow multiple network to be created from a single network address.

REQUIREMENT:

cisco packet Traces

Routers
switches
PCs
Copper straight-through cables.

PROCEDURE:

1) Open cisco packet Traces and create a new network topology.
2) Add the required routers, switches and PCs to the workspace.
3) Connect the PCs to switch to the router using suitable cables.
4) Divide the network 192.168.1.0/24 into smaller subnet using the 127 subnet mask.
5) Assign static-IP address to all PCs and configure the default gateway
6) Configure the router interface through the CLI

Example commands:

```
Enable
configure terminal
IP address 192.168.1.1. 255.255.255.224
no shutdown
exit
interface FastEthernet 0/1
IP address 192.168.2.1. 255.255.255.224
no shutdown
exit
```

7) Configure IP address, subnet mask and default gateway on each PC.

8) Check the connection between PCs using ping command.

9) Send a simple PDU between PCs in different subnets.

10) Observe packet transmission with and without routes

Device	IP Address	Subnet mask	Gateway
Router	192.168.1.1	255.255.255.224	—
PC 1	192.168.1.11	255.255.255.224	192.168.1.1
PC 2	192.168.1.12	255.255.255.224	192.168.1.1
PC 3	192.168.1.13	255.255.255.224	192.168.1.1

Result

Thus, subnetting was successfully implemented and cisco packet traces.

EXPERIMENT - 10

Internetworking with routers in CISCO packet Tracker Simulator

AIM.

To design and configure a simple internetwork using a router and establish communication between two different network.

REQUIREMENT:

Cisco Packet Traces
1 Router
2 PCs
Copper straight through cables.

THEORY:

A router is a networking device used to connect different network and foward packet between them. In two PCs are connected to two different interface of routers static IP address are assigned to PCs and router interfaces.

IP configuration:

Device	IP address	Subnet mask	Gateway
Router Fa 0/0	192.168.10.1	255.255.255.0	—
Router Fa 0/1	192.168.20.1	255.255.255.0	—
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Procedure

1) Open cisco packet tracer
2) Place one router and two PCs in workspace
3) Connect PC0 to Router 1 FastEthernet 0/0 using copper straight through cable
4) Connect PC1 to Router 1 FastEthernet 0/1 using copper straight through cable
5) Open router CLI and enter
6) Configure PC0:

   IP : 192.168.10.2

   Subnet mask : 255.255.255.0

   Default Gateway : 192.168.10.1

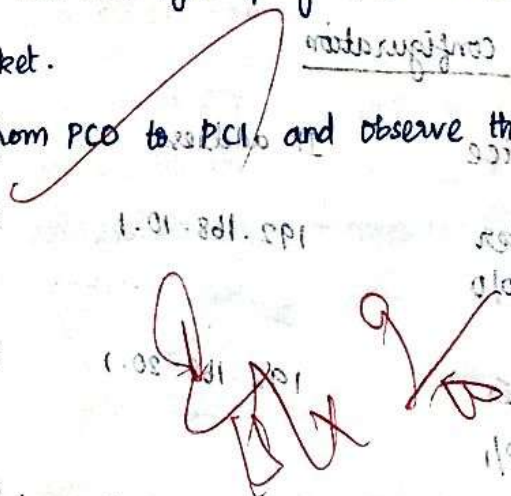
   Configure PC1

   IP : 192.168.20.2

   Subnet mask : 255.255.255.0

   Default Gateway : 192.168.20.1

7) Open command prompt PC0 and type ping 192.168.20.2
8) Observe the reply packet.
9) Send a simple PDU from PC0 to PC1 and observe the packet transmission.



Result:

Thus, the internetwork was successfully assigned and configured using a router in cisco packet Tracer

EXPERIMENT - 11

Routing at Network Layer

AIM:

To simulate static routing and RIP using cisco packet traces.

A) Static Routing

Static routing is a method where router are manually added to the router routing table. It is useful for controlling path taken by packet between different network.

Procedure:

1) Open cisco packet traces
2) Place required routers, switch and PC in workspace.
3) Connect the device using suitable cables
4) Assign IP address to all PCs and routers interface.
5) Open the router CLI.
6) Enter the foll command to configure static route
7) Use foll command to check routing table. Show IP route.
8) Use ping command to check connectivity between PCs.

B) RIP Routing:

RIP is a dynamic routing protocol. It allows routers to automatically exchange information about the network connected to them. RIP select routes based mainly on no of hops.

Procedure:

1) Create network with three routers and PCs shown in packet traces topology.
2) Assign IP addresses to PCs and router interfaces.
3) Configure the router interface using
4) Enable RIP on routes
5) Repeat RIP configuration for other routers with their directly connected network.

6) Allow some time for routers to exchange routing information

7) Check the routing table using
   Show IP route

8) Test communicating using
   Ping <destination IP>

9) Send simple PDU from source PC to destination PC and observe packet path.

Observation:

In static routing, routers are manually configured by administrator.

In RIP routers automatically exchange routing information.

RIP selects a route based on lowest hop count

Result:

Thus, static routing and RIP were successfully simulated using cisco packet tracer.

EXPERIMENT- 12

End to end communication at Transport Layer

AIM:

To implement an echo-client server using TCP/UDP socket and implement a chat client-server using socket programming.

a) Echo-client server using TCP/UDP

A socket is an endpoint used for communication between two device. In an echo program, the client sends a message to server and the server sends the same message back to client.

Procedure:

1) Create server program and client program.
2) Import the required socket library.
3) In server, create a socket and bind it to an IP address and port no.
4) Make the server listen to the client request.
5) Start client and connect it to server.
6) Enter message from client.
7) Send message to serve
8) The server receives the message and send same message back.
9) The client receives and display the echoed message.
10) Close socket connection.

b) Client server using socket program

1) Create server and client program using socket programming
2) Start server and bind it to an IP address and port.
3) Make server wait for client connection.
4) Start client and connect it to server.
5) Enter message in client.

6) Send message to server
7) The server receives and display message.
8) The server sends a reply back to client.
9) Continue sending and receiving message to perform a chat.
10) Close the connection after the chat is completed.

Result:

The end to end communication was successfully implemented using TCP/UDP sockets.

EXPERIMENT- 13

Implement your own ping program

AIM:

To Implement a simple ping program to check the connectivity between two network devices.

THEORY:

Ping is a network utility used to check whether a destination device is reachable through a network. It sends a request to the destination and waits for a reply. If reply is received, connection is successful.

REQUIREMENTS:

Computer
Python / suitable programming environment
Network connection

PROCEDURE:

1) Create a new program for implementation ping operation.
2) Import the required socket / network library.
3) Get IP address or hostname of the destination
4) Create a socket for communication
5) Send required packet to the destination
6) Wait for response from destination
7) If a response is received, display that destination is reachable.
8) Reach the process for required number of packets.
9) Display the result and close the connection.

ALGORITHM:

1) Start
2) Enter destination IP address
3) Create a socket
4) Send a ping request
5) Wait for response
6) If response is received
7) Otherwise display host is unreachable
8) Stop

RESULT:

Thus, the ping program was successfully implemented.

EXPERIMENT - 14

Raw sockets - packet Sniffing

AIM:

To write a program using RAW sockets to implement packet sniffing.

THEORY:

A packet sniffer is a program used to capture and examine network packets travelling through a network interface. A raw select allows program to access network packets directly at lower level, packet information such as source address, destination address.

REQUIREMENTS:

Computer / Laptop
Python
RAW socket support
Network interface
Administrator / Root privileges.

ALGORITHM:

1) Start the program
2) Create RAW socket
3) Find the socket to required network interface
4) Wait for incoming network packets
5) Capture the packet using RAW socket
6) Extract packet information
7) Display packet detail
8) Stop program when required

PROCEDURE:

1) Open python IDE or terminal
2) Create a python program for packet sniffing
3) Import socket module
4) Create RAW socket.
5) Use receivefrom() fun to receive packets
6) Road captured packet data
7) Display packet information
8) Generate some network traffic and observe the captured packets.

RESULT:

Thus, a RAW socket based packet filling program was implemented successfully.

EXPERIMENT - 15

Analysis of web logs using webalizer tool

AIM:

To analyze the different types of web logs using webalizer tool.

THEORY:

Webalizer is a web log analysis tool used to process web server log files and generate report.

REQUIREMENTS:

Computer / Laptop
Webalizer window version
Web log file

PROCEDURE:

1) Run the webalizer window version
2) Select or provide the required web log file
3) Download web log file from web if required
4) Load the web log file into webalizer.
5) Press Run Webalizer
6) Webalizer process the log file
7) Observe the generated analysis / report
8) study the different information available on the report.

INPUT:

A web server log file

OUTPUT:

Web log analysis report generated using Webalizer

RESULT:

Thus, the weblog file was successfully analyzed using webalizer tool.